Video Transcript

Video 8: Creating a Workflow in Airflow: Directed Acyclic Graphs (DAGs) (03:56)

Writing a workflow in Airflow is very similar to simply writing a Python program. In fact, what you end up is a Python file. Typically, you do your imports, then you do a number of tasks, and then you clean or close somehow. Now, when it comes to writing workflows for Python, you have that Python file that I mentioned, but you need to give it some structure so Airflow knows what you're trying to do. You start off with the imports as you usually do. Then, you'd have your DAG arguments that you're going to pass into your DAG instance, your tasks, and some way of expressing your dependencies. So, let's take a look at each one of these in turn. We'll start off by looking at the imports. You'd start off by pulling in Airflow, as you tend to do whenever you're working with a new package. Then, you would define your operators, we will need these when we define our tasks. You can see there that we're going to be using a bash operator and a Python operator.

Now moving forward, we have our default arguments. As you can see there, many of these map to the usual concerns that you have whenever you're writing a workflow using scripts, using glue code from the ground up. You have your owners who you notify in case of a problem, how many retries you should have, and when you end and many others that you can see there. Now when it comes to instantiating the DAG, you start off by giving it a name. You'd also add a description, you'd pass in your default arguments and then your schedule interval. In this case, I'm making it here once a day and I wanted to start off two days back. It'll try to catch up by default, which means that as soon as I turn it on in this case, it would run a couple of times. Now, when it comes to expressing time, here are a number of presets that you can use as well as you can also use the cron type of notation. Now, when it comes to the tasks themselves, you can see here that t1, t2, and t3 are tasks and they are Python dictionaries. And note that we are instantiating for each of those the bash operator and then the Python operator.

In each of these, in the first two, we're running a bash command, you can see there that the first one is a date, then on the second one we are sleeping, and then on the third one we're simply calling a function with Python callable and that function is there at the top of this code block

© MIT xPRO® 2024. All rights reserved.

where it says sample function, and then in this case, we would simply return a 5. Now, you can see this if you run it on the workflow platform and you look at the logs, you'd be able to see that output. The only remaining thing to do from that list that we took a look at of the number of things that you need to define is to set your dependencies, the order in which to execute the tasks. And you can see here that you could add a t2 to depend on t1 and you would specify it as shown. And, you could use the a different notation for the same where you have perhaps a more transparent way where you have t1 followed by t2 and between them you have your greater than less than type signs, in this case. Now, you could also specify brackets to be able to define tasks that will run in parallel. In this case, you can see here that t2 and t3 would run in parallel after t1 has completed. So, that defines the makeup of the number or the required structure for creating a DAG, and ultimately, as mentioned, it is simply a Python file.

© MIT xPRO® 2024. All rights reserved.